Natural Language Processing

Programming Task-2

Custom Text Generation and GPT-2 Fine-Tuning

Saggu Nagaraju

Task 1 - Customized Text Production Using Decoding Strategies

The first part of the project implements a custom decoder for GPT-2 text generation. The decoder loads the GPT-2 tokenizer and language model, prepares the selected computing device, configures padding when required, and generates text one token at a time. The main generation logic is implemented in the _sample_token method.

Sampling Strategies Tested

Strategy	Configuration	Purpose
Greedy decoding	do_sample = False	Selects the highest-probability token at every generation step.
Temperature sampling	temperature = 0.5	Adjusts randomness in the probability distribution.
Top-k filtering	k = 10 and k = 50	Limits candidates to the most probable tokens.
Top-p sampling	p = 0.5 and p = 0.95	Samples from tokens within a cumulative probability mass.
Hybrid sampling	temperature = 0.7, top_k = 50, top_p = 0.9	Combines multiple controls for flexible generation.

Generation Quality Observations

Sampling Strategy	Observed Output Quality
Greedy	Stable but repetitive; often continued the prompt structure.
Temperature = 0.5	More controlled than open sampling, but still unstable.
Top-k = 10	Restricted vocabulary; sometimes concise but often fragmentary.
Top-k = 50	More varied than top-k 10, but less consistent.
Top-p = 0.95	Higher variation, with more off-topic continuation.
Hybrid	Flexible generation, but quality depends on model alignment.

Task 1 Conclusion: The decoding experiments show that generation parameters strongly affect text stability and variation. Greedy decoding is predictable but repetitive, while top-k, top-p and hybrid sampling improve diversity at the risk of noisy continuation.

Task 2 - Fine-Tuning GPT-2 on Instruction Dataset

The second part of the project fine-tunes GPT-2 on Alpaca-style instruction data. Causal language modeling is used, where the labels are equal to the input token IDs. During training, the model performs a forward pass, computes language modeling loss, applies backpropagation, clips gradients, updates the optimizer and follows a linear learning-rate schedule.

Fine-Tuning Setup

Parameter	Value
Model	GPT-2
Dataset	Alpaca instruction-following dataset
Training subset	Small dataset mode with 10 examples
Train / test split	8 training examples and 2 test examples
Epochs	1
Batch size	4
Learning rate	5e-5
Warmup steps	50
Maximum length	128 tokens
Loss	Causal language modeling loss

Training Result

- The recorded first-batch loss was 5.6799.
- The average loss after the single training epoch was 5.2044.
- The model and tokenizer were saved after fine-tuning for later generation and evaluation.

Quantitative Results

Metric	Value
Perplexity	277.3959
Average repetition score	0.2801
Average diversity score	0.4888
Average Jaccard similarity	0.5177

- High perplexity indicates that instruction-following behaviour is difficult to learn from a very small dataset.
- The repetition score reflects repeated phrases and prompt markers in several generated responses.
- The diversity and Jaccard similarity scores show moderate variation and partial overlap with expected text.

Qualitative Examples and Discussion

The generated output file shows that the fine-tuned model can answer several simple instructions, but the responses still include repetition and prompt-continuation artifacts.

Prompt Type	Observation
Health tips	The output included useful advice, but repeated similar health tips several times.
Air pollution	The response was mostly relevant and mentioned renewable energy, public transport and emission policies.
Atom structure	The explanation started coherently, but later continued by repeating the prompt format.
Grammar correction	The corrected sentence appeared, followed by repeated error messages.
Primary colors	The first answer was correct, but later repeated answers became inconsistent.
3D model request	The model produced a no-output style response and repeated that response.

Strengths

- The custom decoder supports multiple generation strategies in a single implementation.
- The fine-tuning pipeline completes tokenization, training, generation and evaluation as one workflow.
- Logs and generated output files make the experiment traceable and reproducible.

Limitations

- The dataset subset is very small, limiting instruction-following performance.
- Only one training epoch was used, so improvement over the base model remains limited.
- Several outputs contain repetition, noisy tokens and prompt-format continuation.

Conclusion

This project demonstrates how GPT-2 can be controlled through decoding strategies and adapted through fine-tuning on instruction-style data. The custom decoder shows that generation parameters have a direct impact on response diversity and stability. The fine-tuning experiment improved the structure of several instruction-style outputs, while the very small dataset and short training setup resulted in high perplexity and repeated text.

Overall, the implementation provides a clear practical foundation for understanding language-model generation, decoding controls, fine-tuning pipelines and text-quality evaluation. Future improvements should include a larger training subset, more epochs, validation-based model selection and stronger generation stopping rules to reduce prompt continuation and repetition.

Project Outcomes

Outcome Area	Achievement
Custom decoding	Implemented greedy, temperature, top-k, top-p and hybrid text generation controls.
Fine-tuning	Adapted GPT-2 to Alpaca-style instruction prompts using causal language modeling.
Evaluation	Measured perplexity, repetition, diversity and lexical overlap of generated outputs.
Practical learning	Established a working pipeline from data loading to generation and result analysis.